

PayMind: a scalable tool-calling agent for PayPal

Design document · Datazoic programming task

Author Akshayaa D **Code** github.com/akshayaad19/paymind

Stack LangGraph · Gemini · Qdrant (hybrid search) · FastAPI · LangSmith

In one paragraph. PayMind turns the PayPal Postman collection into **112 searchable tool cards** and lets a single LangGraph agent use them from plain English. Instead of giving the model every tool, each message first runs a **role-filtered hybrid search** (BM25 + dense vectors, merged with RRF) and offers the model only the **top 5**, plus a few always-available built-ins, including the required **RAG pipeline tool** and **System Search tool**. Every call the model proposes goes through **code** (validation, permissions, confirmation for writes) before it runs, and every step is traced in **LangSmith**. Measured on 1,012 tools (112 real + 900 deliberately confusing look-alikes), the right tool stays in the top 5 for **100% of test questions** at about **20 ms** per search.

1. What the brief asked for, and where it is

Requirement	How PayMind meets it
Agentic system over the PayPal Postman collection (50+ APIs)	112 requests parsed and enriched into tool cards; a mock PayPal server runs them (30 endpoints with real state in SQLite, the rest replay PayPal's example responses).
"Send an invoice for \$50 to..."	Agent chains <code>create_draft_invoice</code> → <code>send_invoice</code> ; both confirmed by the user; tested live.
"Total sales volume last month?"	<code>list_transactions</code> over the date range; the model sums the sales.
"Dispute open from user_123?"	<code>list_disputes</code> , then results are filtered to what the caller may see.
RAG Pipeline Tool	<code>rag_search</code> : 15 policy documents (shop + PayPal) → 336 chunks → hybrid search + cross-encoder reranker; answers only from the passages, with citations.
System Search Tool	<code>system_search</code> : what the assistant can do (searches the tool index) and the user's own recent activity (audit log); <code>find_tools</code> searches the whole catalogue.
Scale from 100+ to 1000s of tools	Retrieval-first tool selection (§4), measured at 112 / 512 / 1,012 tools (§8).
Framework choice, observability	LangGraph, justified in §9; LangSmith traces every node, search, validation and API call (§7).
Agent structure, routing, state, scalability, error handling	§3, §4, §5, §8, §6.

Beyond the brief, the demo is a working **seller's portal** for one shop: an accountant (Asha) and customers (Rahul, Priya) log in (JWT), chat with the agent, handle disputes end to end (messages, photos, refunds, replacements, tracking) and purchase orders (including handwritten ones read by Gemini). 231 automated tests run without any LLM.

2. Architecture

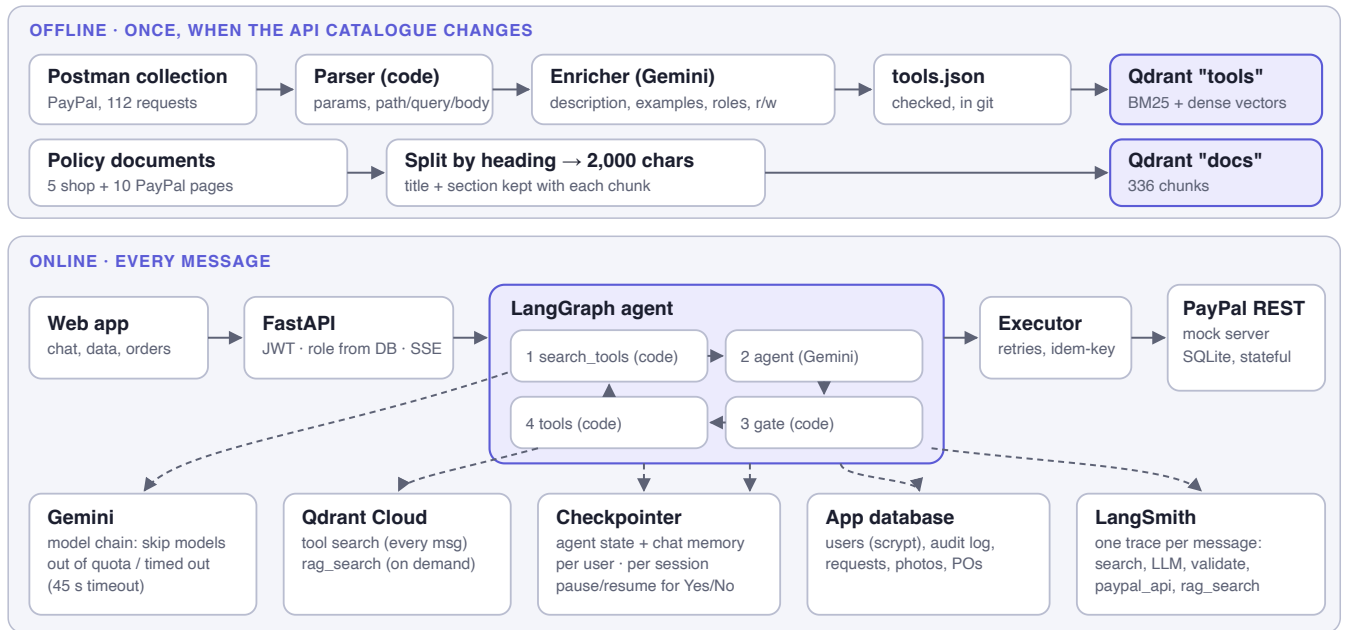


Figure 1. Offline, the Postman collection becomes validated, searchable tool cards (the enricher's output is schema-checked and every example call is dry-run against the mock). Online, only code talks to PayPal: the model proposes, the gate checks, the executor runs.

3. Agent structure

One **ReAct** agent (reason → act → observe, repeated) built as a LangGraph state machine. Only one of the four nodes is an LLM; everything that has a single right answer is code.

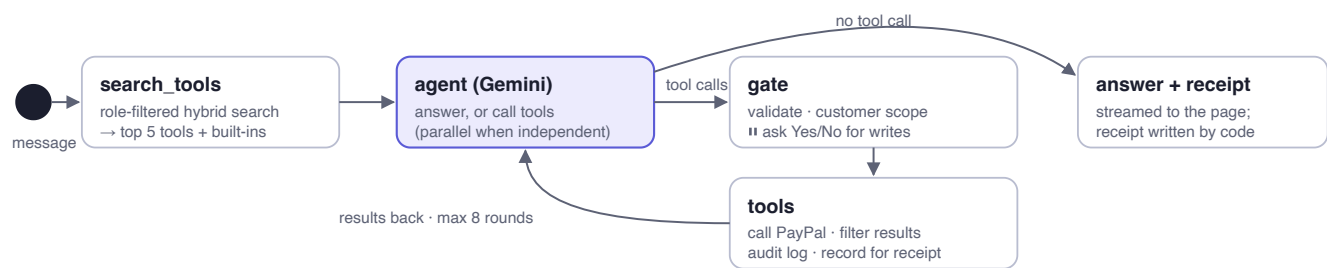


Figure 2. The loop. The model sees only the tools it was offered; nothing it asks for runs without passing the gate.

Node	LLM?	Responsibility
search_tools	No	Picks the tools for this message (§4). Short follow-ups ("refund the first one") are searched together with the previous message.
agent	Yes	Decides the next step from the conversation, tool results and errors. Can run independent calls in parallel, or chain dependent ones (list → details → refund).
gate	No	Validates every proposed call (§6); for writes, pauses the graph with <code>interrupt()</code> and asks the user. Changes nothing, so a resumed run never repeats an action.
tools	No	Executes only approved calls, filters list results to the caller's own records, writes the audit log, and records what really happened.

Why one agent and not several? Tool search already does the routing a supervisor agent would do, for one LLM call fewer per message. Safety checks stay in deterministic code, which a second "validator" LLM could not guarantee. The graph is modular, so domain sub-agents (disputes, invoicing, reporting) can be added as sub-graphs if the catalogue outgrows a single prompt (§8).

4. Tool selection and routing

Every tool card is indexed twice in Qdrant, from its name, category, description and four example questions:

- **BM25** (sparse): exact words and IDs, e.g. "refund", "PP-D-38106".
- **Dense** (bge-small, 384-d): meaning, e.g. "money back" ≈ refund.

Each returns its top 20; **Reciprocal Rank Fusion** merges them by rank and the top 5 go to the model. The user's role is a **payload filter inside both searches**, so a customer is never even shown a refund tool.

Why this scales: the prompt size is constant (5 + built-ins) whatever the catalogue size; adding an API means adding a card and re-indexing, with no prompt changes. What the model reads is engineered: each tool shows its required fields and a validated example call, results are compacted, and errors are one plain sentence.

Always offered (built-ins, role-dependent):

- `rag_search` : policy questions (the RAG pipeline tool).
- `system_search` : capabilities and the user's own activity.
- `find_tools` : searches the full catalogue when the 5 don't fit.
- `check_updates` , `order_status` : what's new, purchase orders.
- Customer: report a problem, ask for refund/replacement, close a case. Shop: find a customer and their payments.

RAG pipeline tool

Documents are split by Markdown heading, then to 2,000 characters (200 overlap), keeping "title › section" with every chunk. Search is the same hybrid + RRF (top 20), then a **cross-encoder reranker** reads question and chunk together and keeps the best 5; weak matches are marked, so the agent says "not in the policies" instead of guessing. Shop policy is also **enforced in code** where money is involved (e.g. in-store purchases: at most 50% back).

5. State management

What	Where and why
Agent state	LangGraph <code>AgentState</code> : <code>messages</code> (appended), <code>offered</code> tools, <code>seen</code> (text used to check IDs are real), <code>steps</code> , <code>decisions</code> (gate verdicts), <code>actions</code> (what really ran). Each node returns only the fields it changes.
Memory and pause/resume	A checkpointer saves the state after every node, per <code>user_id__session</code> . A confirmation pauses the run; the user's Yes resumes it exactly where it stopped, even in another HTTP request.
Identity	JWT carries only who you are; the role is read from the database on every request, and a token version makes logout revoke every copy of the token.
Business data	PayPal (mock) is the source of truth for payments, invoices, disputes and tracking. The app database holds what PayPal has no concept of: users, audit log, purchase orders, refund/replacement requests, photos.

6. Error handling and safety

Layer	What it catches
Validator (5 checks, code)	Tool allowed and offered for this role · parameters match the JSON Schema (obvious type slips fixed) · business rules (amount > 0, 2 decimals, USD, valid emails) · grounding : every ID must have appeared in the conversation (no invented IDs) · writes need confirmation, with a stronger warning over \$500. Failures go back to the model to fix.
Scope and money rules (code)	Customers only touch their own records; a payment with an open dispute can't be refunded directly; refunds can't exceed what's left; the confirmation names the real customer and purchase, looked up from PayPal, not the model's words.
Executor	Retries timeouts, 429 and 5xx (1 s, 2 s, 4 s) with the same PayPal-Request-Id , so a retried refund can never happen twice; 4xx errors become one sentence for the model.
Model failures	Model chain: a model out of daily quota is skipped until tomorrow; a timed-out or overloaded one is skipped for 5 minutes; 45 s timeout per call; clear message if all fail.
Truth to the user	Under every reply, a receipt written by code from what actually ran ("✅ Done: Refund 79.99 USD" or "No changes were made"), so a model that claims an action it didn't take is caught immediately. A step limit (8) stops runaway loops and says what was done.

7. Observability

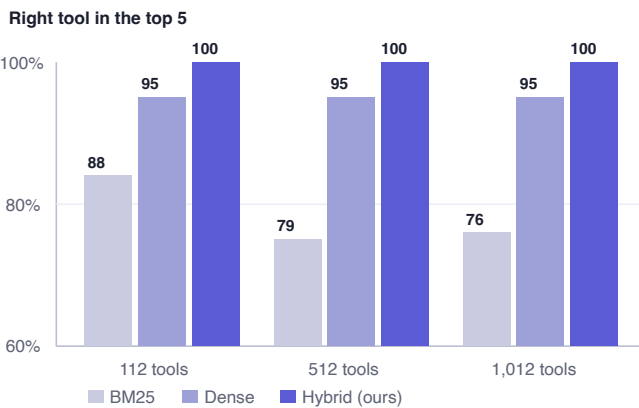
Each message is one LangSmith trace (`paymind_chat`), tagged with user, role and session. Its spans show the **tool search** (tools, scores, allowed roles), each **Gemini call** (prompt, tool choice, model, tokens), the **validator** verdict, every **PayPal call** (status, attempts, whether the role was allowed) and **RAG** hits. So any answer can be explained: what the model saw, what it decided, what code allowed, what ran, and how long each took. The in-app audit log keeps the same actions for users; developer detail stays in LangSmith.

8. Scalability

Measured: does tool search still find the right tool at 1,000 tools?

42 hand-written questions (not from the tool cards) were searched over the 112 real tools, then with 400 and 900 synthetic tools added from 25 look-alike services (other payment, billing, CRM, accounting, shipping, support APIs) that reuse PayPal's vocabulary ("refund a charge", "list invoices", "respond to dispute") to be as confusing as possible.

Tools	Search	Top-5	Top-1	ms
112	BM25	88%	60%	10
112	Dense	95%	83%	10
112	Hybrid	100%	79%	21
512	BM25	79%	52%	9
512	Dense	95%	83%	10
512	Hybrid	100%	76%	20
1,012	BM25	76%	52%	9
1,012	Dense	95%	79%	13
1,012	Hybrid	100%	71%	23



Reading: keyword search alone degrades as look-alikes are added (88% → 76%); dense holds at 95%; the hybrid keeps the right tool in what the model sees **every time**, at ~20 ms. Top-1 falls from 79% to 71%, which is why the model is shown 5 candidates, not 1, and chooses among them with their descriptions. Honest limits: 42 questions, template-generated distractors, local in-memory Qdrant. Reproduce with `python -m paymind.retrieval.eval_tools`.

Going to 10,000+ tools and many users

Pressure	Plan
Tool catalogue	Keep retrieval-first. Add a cross-encoder reranker on the top 20 tools (already used for RAG) to lift top-1; a service/category pre-filter as a payload filter; find_tools remains the fallback. Onboarding a new API is the same offline pipeline: parse → enrich → validate → index.
Very different domains	A router picks a domain, then a domain sub-agent (its own tools, prompt, rules) runs as a LangGraph sub-graph. Only worth it when one prompt can't hold the rules; tool search covers routing until then.
Traffic	The API is stateless (JWT), so it scales horizontally. Move the checkpointer and app data from SQLite to Postgres , add Redis for the per-user role cache (short TTL, cleared on change), run Qdrant as a cluster.
LLM cost and limits	Only one LLM call per round; small tool list keeps prompts short. Paid tier with the same model chain for fallback; cache policy answers.
Quality over time	The eval above becomes a regression test in CI; LangSmith traces feed new test questions from real failures.

9. Why these technologies

Choice	Why	Considered
LangGraph	Explicit graph: code decides where the LLM is used, where checks happen, and how loops end. Built-in <code>interrupt()</code> for human approval, checkpointing for memory and resume, streaming, and native LangSmith tracing.	CrewAI / AutoGen: role-play agents, less control over each step. A hand-written loop: would need pause/resume and persistence rebuilt.
Qdrant	Sparse (BM25 with IDF) and dense vectors in one collection, server-side RRF fusion, payload filters inside the search (role), local mode for tests and a free cloud tier.	FAISS: dense only, no filters. Chroma: no native sparse + fusion.
fastembed	Small ONNX models run locally: embedding, BM25 and the reranker with no API cost and no data leaving the machine.	Hosted embedding APIs.
Gemini	Good tool calling, multimodal (reads handwritten purchase orders), free tier for development; wrapped in a fallback chain so any model can be swapped.	Any tool-calling model fits the same interface.
LangSmith	Traces LangGraph runs node by node with no extra plumbing; custom spans for search, validation and API calls.	Langfuse, OpenTelemetry.

10. Limitations and next steps

- PayPal is a **mock server** (real request/response shapes, 30 stateful endpoints); the next step is the PayPal sandbox with OAuth.
- The scaling eval is small (42 questions); grow it from real traces and run it in CI. Add the tool reranker and measure top-1.
- Production: Postgres instead of SQLite, HTTPS, secrets manager, rate limiting, and LLM-call evaluation (tool-choice accuracy, end-to-end task success).